Statically reasoning about R vectors

Designing an abstract domain for a real language

Manuel Di Agostino[1], Florian Sihler[2], Vincenzo Arceri[1], Oliver Gerstl[2], Matthias Tichy[2]

28 October 2026

[1] University of Parma, [2] Ulm University
Static Analysis and Software Verification, Seminar

manuel.diagostino@unimore.it

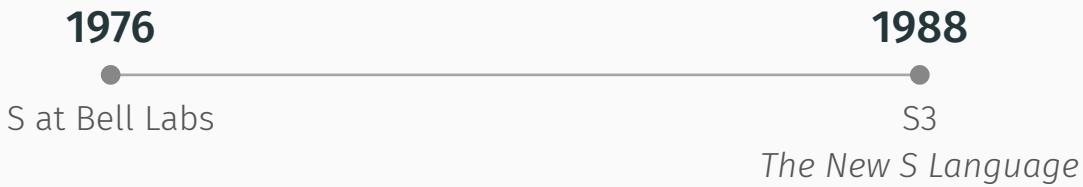
M. Di Agostino, F. Sihler, V. Arceri, O. Gerstl, M. Tichy. *Towards Statically Reasoning about R Vectors*. NSAD '26, pp. 18–25. doi:10.1145/3840563.3843708

Roadmap

R → μR → Domain → Operators → Validation

1. From S to R

1976 → 1988



Batch (Fortran)

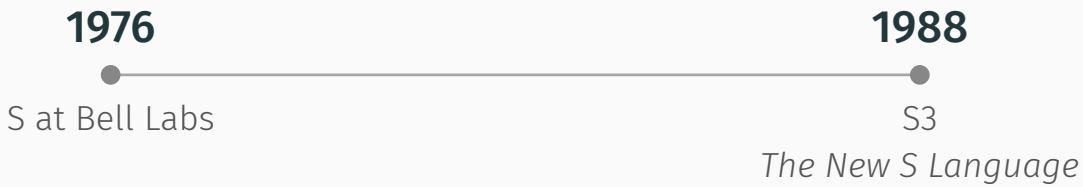
```
      PROGRAM ANALYSIS
      READ (5, *) N, (X(I), I = 1, N)
      CALL MEAN(X, N, AVG)
      WRITE (6, *) AVG
      STOP
      END
C     unit 5 = card reader
C     unit 6 = line printer
```

edit → submit → queue → print, in **hours**

S: an interactive environment for statisticians.[1]

[1] Dates: R. Ihaka, *R: Past and Future History*, 1998; R FAQ §3.3.1.

1976 → 1988



Interactive (S)

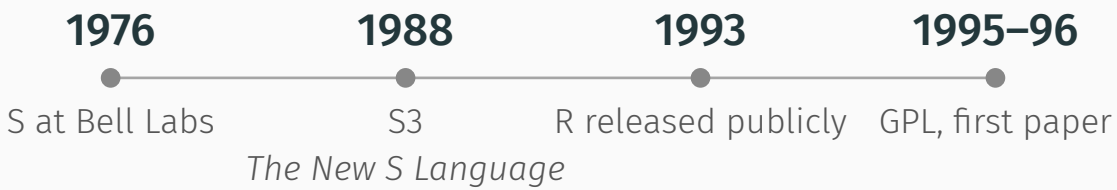
```
x <- c(3.1, 4.2, 5.0)
mean(x)          # whole column
plot(x, y)
lm(y ~ x1 + x2)  # formula
```

type → see → adjust, in **seconds**

S: an interactive environment for statisticians.[1]

[1] Dates: R. Ihaka, *R: Past and Future History*, 1998; R FAQ §3.3.1.

1976 → 1996



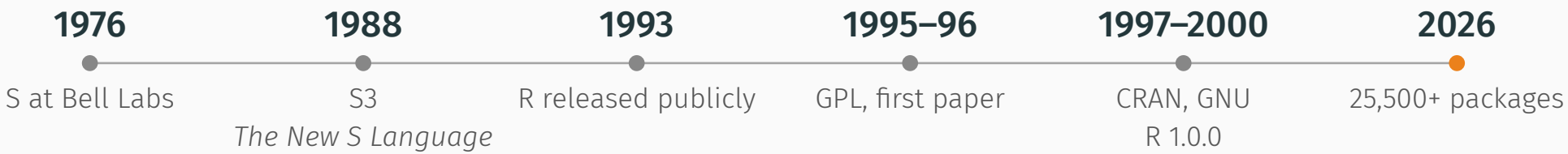
Pipeline (R)

```
read.csv("cars.csv") |>
  subset(speed > 10) |>
  transform(time = dist / speed) |>
  plot(time ~ speed, data = _)
```

from file to figure, in one expression

R: a free S, built by statisticians.

1976 → 2026



The ecosystem grew faster than the tooling.[1]

[1] Dates: R. Ihaka, *R: Past and Future History*, 1998; R FAQ §3.3.1.
Package count: CRAN, *Contributed Packages*, accessed 25 Oct 2026: 25,529.

The tooling gap

Correctness = plots and tests

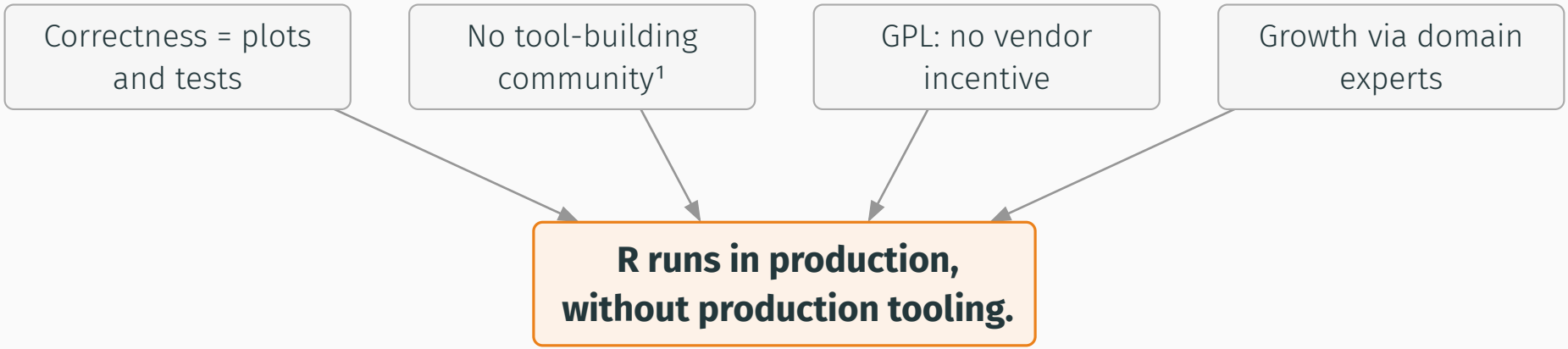
No tool-building community[1]

GPL: no vendor incentive

Growth via domain experts

[1] J. Fox, *Aspects of the Social Organization and Trajectory of the R Project*, The R Journal 1(2), 2009.

The tooling gap



[1] J. Fox, *Aspects of the Social Organization and Trajectory of the R Project*, The R Journal 1(2), 2009.

2. Why R Resists Static Analysis

Everything is a vector

Everything is a vector:

- `TRUE`, *logical* of length 1
- `3` or `3.14`, *double* of length 1
- `c(1L, 2L, 3L)`, *integer* of length 3
- `"hello"`, *character* of length 1

There is no scalar type.

Everything is a vector

Everything is a vector:

- `TRUE`, *logical* of length 1
- `3` or `3.14`, *double* of length 1
- `c(1L, 2L, 3L)`, *integer* of length 3
- `"hello"`, *character* of length 1

There is no scalar type.

`NA` follows the same rule. It is the sentinel for a missing value: it occupies a position and it propagates.

Everything is a vector

UNIVERSITÀ DI PARMA

Everything is a vector:

- `TRUE`, *logical* of length 1
- `3` or `3.14`, *double* of length 1
- `c(1L, 2L, 3L)`, *integer* of length 3
- `"hello"`, *character* of length 1

There is no scalar type.

`NA` follows the same rule. It is the sentinel for a missing value: it occupies a position and it propagates.

```
length(NA)          # 1             one element, and it is missing
c(1, 2, NA)         # [1, 2, NA]    NA occupies position 3
mean(c(1, 2, NA))   # NA            and it propagates
```

NA is not NULL

The two “nothings” of R are different objects.

`NA`: the value is unknown

- it is an element: it sits at a position
- any operation on it yields `NA`

```
x <- c(1, NA, 3)
length(x)        # 3
sum(x)           # NA
```

NA is not NULL

The two “nothings” of R are different objects.

NA: the value is unknown

- it is an element: it sits at a position
- any operation on it yields NA

```
x <- c(1, NA, 3)
length(x)        # 3
sum(x)           # NA
```

NULL: there is no value

- it is not an element: it has no position
- it disappears from a vector, so the length shrinks

```
y <- NULL
length(y)        # 0
c(1, NULL, 3)    # [1, 3]
```

NA is not NULL

The two “nothings” of R are different objects.

NA: the value is unknown

- it is an element: it sits at a position
- any operation on it yields NA

```
x <- c(1, NA, 3)
length(x)        # 3
sum(x)           # NA
```

NULL: there is no value

- it is not an element: it has no position
- it disappears from a vector, so the length shrinks

```
y <- NULL
length(y)        # 0
c(1, NULL, 3)    # [1, 3]
```

NULL is the empty vector, and `c()` is its literal. NA is a value that says missing.

→ The analysis must know: how long is this value, and what sits at each position?

Silent coercion

Remark Type Hierarchy

Vectors elements must be homogeneous in their type:

$$\texttt{logical} < \texttt{integer} < \texttt{double} < \texttt{character}$$

Implicit coercion occurs: when types are mixed, the elements are promoted to their least upper bound.

Silent coercion

Remark Type Hierarchy

Vectors elements must be homogeneous in their type:

$$\text{logical} < \text{integer} < \text{double} < \text{character}$$

Implicit coercion occurs: when types are mixed, the elements are promoted to their least upper bound.

```
c(TRUE, "hello")   # ["TRUE", "hello"]
```

Silent: no error, no warning, and nothing in the source says `character`.

Silent coercion

`NA` exists once per type. It prints without quotes, so it is not the string `"NA"`:

```
c(NA, "hello")     # [NA, "hello"]
```

Silent coercion

`NA` exists once per type. It prints without quotes, so it is not the string `"NA"`:

```
c(NA, "hello")     # [NA, "hello"]
```

The result takes the least upper bound of the element types:

```
c(TRUE, 3L, 2.5)   # [1.0, 3.0, 2.5]    double
1L + TRUE          # 2L                 integer
```

Silent coercion

`NA` exists once per type. It prints without quotes, so it is not the string `"NA"`:

```
c(NA, "hello")      # [NA, "hello"]
```

The result takes the least upper bound of the element types:

```
c(TRUE, 3L, 2.5)    # [1.0, 3.0, 2.5]    double
1L + TRUE           # 2L                 integer
```

→ The analysis must know: which type can each element have?

Recycling

UNIVERSITÀ DI PARMA

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                    # ?
c(0, 0, 0, 0) + c(1, 2)                    # ?
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)        # ?
c(1, 5, 2, 8) > 3                          # ?
c(1, 5, 2, 8) > c(4, 6)                    # ?
```

Recycling

UNIVERSITÀ DI PARMA

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                  # [4, 4, 4]
c(0, 0, 0, 0) + c(1, 2)                  # ?
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)      # ?
c(1, 5, 2, 8) > 3                        # ?
c(1, 5, 2, 8) > c(4, 6)                  # ?
```

Recycling

UNIVERSITÀ DI PARMA

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                 # [4, 4, 4]
c(0, 0, 0, 0) + c(1, 2)                 # [1, 2, 1, 2]
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)     # ?
c(1, 5, 2, 8) > 3                       # ?
c(1, 5, 2, 8) > c(4, 6)                 # ?
```

Recycling

UNIVERSITÀ DI PARMA

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                 # [4, 4, 4]
c(0, 0, 0, 0) + c(1, 2)                 # [1, 2, 1, 2]
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)     # [11, 22, 33, 14, 25, 36]
c(1, 5, 2, 8) > 3                       # ?
c(1, 5, 2, 8) > c(4, 6)                 # ?
```

Recycling

UNIVERSITÀ DI PARMA

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                 # [4, 4, 4]
c(0, 0, 0, 0) + c(1, 2)                 # [1, 2, 1, 2]
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)     # [11, 22, 33, 14, 25, 36]
c(1, 5, 2, 8) > 3                       # [FALSE, TRUE, FALSE, TRUE]
c(1, 5, 2, 8) > c(4, 6)                 # ?
```

Recycling

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                 # [4, 4, 4]
c(0, 0, 0, 0) + c(1, 2)                 # [1, 2, 1, 2]
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)     # [11, 22, 33, 14, 25, 36]
c(1, 5, 2, 8) > 3                       # [FALSE, TRUE, FALSE, TRUE]
c(1, 5, 2, 8) > c(4, 6)                 # [FALSE, FALSE, FALSE, TRUE]
```

Recycling

UNIVERSITÀ DI PARMA

R operations are element-wise across vectors.

```
c(1, 2, 3) + c(3, 2, 1)                 # [4, 4, 4]
c(0, 0, 0, 0) + c(1, 2)                 # [1, 2, 1, 2]
c(1, 2, 3, 4, 5, 6) + c(10, 20, 30)     # [11, 22, 33, 14, 25, 36]
c(1, 5, 2, 8) > 3                       # [FALSE, TRUE, FALSE, TRUE]
c(1, 5, 2, 8) > c(4, 6)                 # [FALSE, FALSE, FALSE, TRUE]
```

When two vectors have different lengths, the shorter is recycled to match the longer: that is why `x > 3` works without writing `c(3, 3, 3, 3)`.

Recycling

When the longer length is not a multiple of the shorter, R warns and computes the result anyway.

Example Not a multiple

```
r <- c(1, 1, 1)
if (runif() >= 0.5) {
  r <- r + c(41, 41, 41, 41)     # [WARN]
                                 # [1, 1, 1] recycled to [1, 1, 1, 1]
} else {
  r <- r - c(1)                  # [1] recycled to [1, 1, 1]
}
```

Recycling

When the longer length is not a multiple of the shorter, R warns and computes the result anyway.

Example Not a multiple

```
r <- c(1, 1, 1)
if (runif() >= 0.5) {
  r <- r + c(41, 41, 41, 41)    # [WARN]
                                # [1, 1, 1] recycled to [1, 1, 1, 1]
} else {
  r <- r - c(1)                 # [1] recycled to [1, 1, 1]
}
```

After the `if-else`, `r` is either [42, 42, 42, 42] or [0, 0, 0].

Recycling

When the longer length is not a multiple of the shorter, R warns and computes the result anyway.

Example Not a multiple

```
r <- c(1, 1, 1)
if (runif() >= 0.5) {
  r <- r + c(41, 41, 41, 41)    # [WARN]
                                # [1, 1, 1] recycled to [1, 1, 1, 1]
} else {
  r <- r - c(1)                 # [1] recycled to [1, 1, 1]
}
```

After the `if-else`, `r` is either [42, 42, 42, 42] or [0, 0, 0].

→ The analysis must know: the branch taken decides the length, not the syntax.

Attributes

Any R object can carry *attributes*: `dim`, `class` and `names` change what it is.

Example From vector to matrix

```
x <- 1:10
dim(x) <- c(2, 5)
#      [,1] [,2] [,3] [,4] [,5]
# [1,]    1    3    5    7    9
# [2,]    2    4    6    8   10
```

Attributes

Any R object can carry *attributes*: `dim`, `class` and `names` change what it is.

Example — From vector to matrix

```
x <- 1:10
dim(x) <- c(2, 5)
#      [,1] [,2] [,3] [,4] [,5]
# [1,]    1    3    5    7    9
# [2,]    2    4    6    8   10
```

The ten integers are unchanged, the shape is not: without `dim` a plain `integer` vector, with it a matrix, `x[1, 2]` a legal index, and `%*%`, `t`, `solve` apply.

From list to data frame

A data frame is not a primitive: it is a named list plus attributes.

```
l <- list(1:3, c("a", "b", "c"))
names(l)                          # NULL
names(l) <- c("id", "label")      # the names become the columns
df <- as.data.frame(l)            # adds class and row.names
#   id label
# 1  1     a
# 2  2     b
# 3  3     c
```

From list to data frame

A data frame is not a primitive: it is a named list plus attributes.

```
l <- list(1:3, c("a", "b", "c"))
names(l)                          # NULL
names(l) <- c("id", "label")      # the names become the columns
df <- as.data.frame(l)            # adds class and row.names
#   id label
# 1  1     a
# 2  2     b
# 3  3     c
```

→ The analysis must know: which attributes are present, not only the elements.

Three indexing modes

The brackets are the same; the index decides the operation.

```r
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]        # ?
v[c(-1, -3)]         # ?
v[c(TRUE, FALSE)]    # ?
```

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]        # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]         # ?
v[c(TRUE, FALSE)]    # ?
```

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]       # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]        # [20, 40]   positions 1 and 3 are removed
v[c(TRUE, FALSE)]   # ?
```

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]       # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]        # [20, 40]   positions 1 and 3 are removed
v[c(TRUE, FALSE)]   # [10, 30]   mask recycled to length 4
```

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]       # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]        # [20, 40]   positions 1 and 3 are removed
v[c(TRUE, FALSE)]   # [10, 30]   mask recycled to length 4
```

- positive: select by position; `0` is skipped, out-of-bounds gives `NA`

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]       # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]        # [20, 40]   positions 1 and 3 are removed
v[c(TRUE, FALSE)]   # [10, 30]   mask recycled to length 4
```

- positive: select by position; `0` is skipped, out-of-bounds gives `NA`
- negative: exclude by position; everything not listed is kept

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]        # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]         # [20, 40]   positions 1 and 3 are removed
v[c(TRUE, FALSE)]    # [10, 30]   mask recycled to length 4
```

- positive: select by position; `0` is skipped, out-of-bounds gives `NA`
- negative: exclude by position; everything not listed is kept
- logical: filter by mask; the mask is recycled to the vector's length

Three indexing modes

The brackets are the same; the index decides the operation.

```
v <- c(10, 20, 30, 40)
v[c(2, 0, 5)]       # [20, NA]   0 is skipped, 5 is out of bounds
v[c(-1, -3)]        # [20, 40]   positions 1 and 3 are removed
v[c(TRUE, FALSE)]   # [10, 30]   mask recycled to length 4
```

- positive: select by position; `0` is skipped, out-of-bounds gives `NA`
- negative: exclude by position; everything not listed is kept
- logical: filter by mask; the mask is recycled to the vector's length

R refuses to mix positive and negative indices; `0` is allowed with either.

→ The analysis must know: the mode of each index and the length of the result.

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # ?
v[-2] <- 5                # ?
v[c(TRUE, FALSE)] <- 0    # ?
v[0] <- 5                 # ?
```

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # [10, 20, 30, NA, 99]
v[-2] <- 5                # ?
v[c(TRUE, FALSE)] <- 0    # ?
v[0] <- 5                 # ?
```

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # [10, 20, 30, NA, 99]
v[-2] <- 5                # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0    # ?
v[0] <- 5                 # ?
```

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # [10, 20, 30, NA, 99]
v[-2] <- 5                # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0    # [0, 20, 0, 5, 0]
v[0] <- 5                 # ?
```

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # [10, 20, 30, NA, 99]
v[-2] <- 5                # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0    # [0, 20, 0, 5, 0]
v[0] <- 5                 # unchanged, and no error
```

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # [10, 20, 30, NA, 99]
v[-2] <- 5                # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0    # [0, 20, 0, 5, 0]
v[0] <- 5                 # unchanged, and no error
```

- past the end: the vector is extended and the gap is filled with `NA`

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99              # [10, 20, 30, NA, 99]
v[-2] <- 5              # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0  # [0, 20, 0, 5, 0]
v[0] <- 5               # unchanged, and no error
```

- past the end: the vector is extended and the gap is filled with `NA`
- a negative index or a recycled mask writes one value to many positions

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99                # [10, 20, 30, NA, 99]
v[-2] <- 5                # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0    # [0, 20, 0, 5, 0]
v[0] <- 5                 # unchanged, and no error
```

- past the end: the vector is extended and the gap is filled with `NA`
- a negative index or a recycled mask writes one value to many positions
- index `0` writes to no position, and R does not complain

Update

Assignment is not local: `v[i] <- x` can grow the vector.

```
v <- c(10, 20, 30)
v[5] <- 99              # [10, 20, 30, NA, 99]
v[-2] <- 5              # [5, 20, 5, 5, 5]
v[c(TRUE, FALSE)] <- 0  # [0, 20, 0, 5, 0]
v[0] <- 5               # unchanged, and no error
```

- past the end: the vector is extended and the gap is filled with `NA`
- a negative index or a recycled mask writes one value to many positions
- index `0` writes to no position, and R does not complain

→ The analysis must know: whether the length is still known after the update.

3. The Analysis Problem and the Soundness Contract

What we want to know

UNIVERSITÀ DI PARMA

At every program point $\ell \in L$[1], we want the set of states the program can be in.

Definition Collecting semantics

A state σ maps each variable to the vector it holds, $\sigma : \mathit{Var} \rightharpoonup \mathcal{V}$, and

$$[\![P]\!](\ell) \stackrel{\text{def}}{=} \{\sigma \mid \text{some run of } P \text{ reaches } \ell \text{ in state } \sigma\}$$

[1] L is the set of possible labels of the program.

What we want to know

UNIVERSITÀ DI PARMA

- For example, given:

```
v <- c(10, 20, 30)
if (runif() >= 0.5) v <- v * 2
```

two states reach the end, one per branch: $\{v \mapsto [10, 20, 30]\}$ or $\{v \mapsto [20, 40, 60]\}$.

What we want to know

UNIVERSITÀ DI PARMA

- For example, given:

```
v <- c(10, 20, 30)
if (runif() >= 0.5) v <- v * 2
```

  two states reach the end, one per branch: $\{v \mapsto [10, 20, 30]\}$ or $\{v \mapsto [20, 40, 60]\}$.
- *Not computable* in general: deciding $\sigma \in [\![P]\!](\ell)$ is deciding reachability, which is *undecidable*.

What we want to know

UNIVERSITÀ DI PARMA

- For example, given:

```
v <- c(10, 20, 30)
if (runif() >= 0.5) v <- v * 2
```

  two states reach the end, one per branch: $\{v \mapsto [10, 20, 30]\}$ or $\{v \mapsto [20, 40, 60]\}$.
- *Not computable* in general: deciding $\sigma \in [\![P]\!](\ell)$ is deciding reachability, which is *undecidable*.
- So we compute an approximation: one abstract state per point (flow-sensitive), one set of vectors per variable (non-relational).

The soundness contract

An analysis is sound when it never loses a run: at every point, the concrete states are among those its abstract state stands for.

Definition — Soundness

$$\forall \ell \,.\, [\![P]\!](\ell) \subseteq \gamma\big([\![P]\!]^\sharp(\ell)\big)$$

The soundness contract

An analysis is sound when it never loses a run: at every point, the concrete states are among those its abstract state stands for.

Definition — Soundness

$$\forall \ell \,.\, [\![P]\!](\ell) \subseteq \gamma\big([\![P]\!]^\sharp(\ell)\big)$$

- γ may be too large, and that costs a false alarm.
- γ may never be too small, because a run outside it is a behaviour the analysis has lost.

The soundness contract

An analysis is sound when it never loses a run: at every point, the concrete states are among those its abstract state stands for.

Definition — Soundness

$$\forall \ell \,.\, [\![P]\!](\ell) \subseteq \gamma\big([\![P]\!]^\sharp(\ell)\big)$$

- γ may be too large, and that costs a false alarm.
- γ may never be too small, because a run outside it is a behaviour the analysis has lost.

The same obligation one level down, on operators:

$$c \in \gamma(v) \Rightarrow \mathrm{op}(c) \in \gamma\big(\mathrm{op}^\sharp(v)\big)$$

What about existing domains and tools?

UNIVERSITÀ DI PARMA

- *FunArray*[1]: uniform segments, no value per position. Recycling forces a choice: unroll every position into its own segment, and the segmentation dissolves, or summarise and lose exactly the facts recycling asks for.
- *Giet et al.*[2]: facts on the whole sequence; recycling needs per-position values and does not preserve sortedness.
- *flowR*[3] — dataflow graphs and slicing, the platform Part 7 is built on; no abstract domain for vector contents.
- *Data frames*[4] — model the transformations of a frame; the vector semantics underneath stays unmodelled.

[1] P. Cousot, R. Cousot, F. Logozzo, *A Parametric Segmentation Functor for Fully Automatic and Scalable Array Content Analysis*, POPL 2011.
[2] J. Giet, F. Ridoux, X. Rival, *A Product of Shape and Sequence Abstractions*, SAS 2023.
[3] F. Sihler, M. Tichy, *Statically Analyzing the Dataflow of R Programs*, PACMPL 9(OOPSLA2), art. 309, 2025.
[4] O. Gerstl, *Tracking the Shape of Data Frames in R Programs Using Abstract Interpretation*, master's thesis, Universität Ulm, 2025

The design brief

Arrived this far, we can sum up what any static analysis of R vectors is obliged to deal with. Taken together, these are the requirements on the domain:

- Length: which lengths are possible, from `NULL` (0) to unbounded.
- Content: which value, and which type, each element can have.
- Operations: which positions survive a selection, and whether an update grows the vector.
- Attributes: present for certain, or only possibly.
- Termination: the length can grow without bound, so the iteration must stop.

4. The *μR* Language

Real R is much wider than vectors:

- environments are first-class values

```
e <- new.env()
e$x <- 1
```

Real R is much wider than vectors:

- environments are first-class values
- arguments are lazy promises

```
f <- function(a, b) a * 10
f(10, stop("boom"))   # 100: b unused
f(stop("boom"), 1)    # Error: a used
```

Real R is much wider than vectors:

- environments are first-class values
- arguments are lazy promises
- code can be rewritten at run time by metaprogramming,

```
eval(quote(`+` <- function(a, b) 42))
1 + 2   # 42
```

Real R is much wider than vectors:

- environments are first-class values
- arguments are lazy promises
- code can be rewritten at run time by metaprogramming,
- every call: R's S3/S4 OOP dispatch.

```r
length.foo <- function(x) 42
v <- c(1, 2, 3)
class(v) <- "foo"
length(v)                # 42, not 3
```

Real R is much wider than vectors:

- environments are first-class values
- arguments are lazy promises
- code can be rewritten at run time by metaprogramming,
- every call: R's S3/S4 OOP dispatch.

μR keeps the constructs that affect vectors and drops the rest:

- `NULL` is not a special object: it is the empty vector.
- An environment is not a value: it is a flat map from variables to vectors.

Syntax

We omit usual `if-else` and `while` constructs for brevity.

$$
\begin{array}{rcl}
E_{idx} & ::= & \varepsilon \\
 & | & E \\
Atm & ::= & Var \mid Float \mid Int \mid Bool \mid \texttt{NULL} \\
Bool & ::= & \texttt{TRUE} \mid \texttt{FALSE} \mid \texttt{NA} \\
\bowtie_{arith} & ::= & + \mid - \mid * \mid / \\
\bowtie_{rel} & ::= & > \mid < \mid = \mid \neq \mid \leqslant \mid \geqslant \\
\bowtie_{log} & ::= & \& \mid | \\
\triangleright & ::= & ! \mid -
\end{array}
$$

$$
\begin{array}{rcl}
E & ::= & Atm \mid \triangleright E \\
 & | & e_1 \bowtie_{arith} e_2 \\
 & | & e_1 \bowtie_{rel} e_2 \\
 & | & e_1 \bowtie_{log} e_2 \\
 & | & \texttt{c()} \\
 & | & \texttt{c}(E_1, \ldots, E_n) \\
 & | & Var \leftarrow E \\
 & | & Var[E_{idx}] \\
 & | & Var[[E_{idx}]] \\
 & | & Var[E_{idx}] \leftarrow E \\
 & | & Var[[E_{idx}]] \leftarrow E \\
 & | & \texttt{setAttr}(Var, \{s_1, \ldots, s_n\}) \\
 & | & \texttt{getAttr}(Var)
\end{array}
$$

`c(...)` $\longrightarrow \oplus$ $\bowtie \longrightarrow$ **Recycle** `v[i]` $\longrightarrow$ **Select** `v[i]` $\leftarrow$ `e` $\longrightarrow$ **Update**

Syntax

We omit usual `if-else` and `while` constructs for brevity.

$$
\begin{array}{rcl}
E_{idx} & ::= & \varepsilon \\
 & | & E \\
 & & \\
Atm & ::= & Var \mid Float \mid Int \mid Bool \mid \texttt{NULL} \\
 & & \\
Bool & ::= & \texttt{TRUE} \mid \texttt{FALSE} \mid \texttt{NA} \\
 & & \\
\bowtie_{arith} & ::= & \texttt{+} \mid \texttt{-} \mid \texttt{*} \mid \texttt{/} \\
 & & \\
\bowtie_{rel} & ::= & \texttt{>} \mid \texttt{<} \mid \texttt{==} \mid \neq \mid \leqslant \mid \geqslant \\
 & & \\
\bowtie_{log} & ::= & \texttt{\&} \mid \texttt{|} \\
 & & \\
\triangleright & ::= & \texttt{!} \mid \texttt{-}
\end{array}
$$

$$
\begin{array}{rcl}
E & ::= & Atm \mid \triangleright E \\
 & | & e_1 \bowtie_{arith} e_2 \\
 & | & e_1 \bowtie_{rel} e_2 \\
 & | & e_1 \bowtie_{log} e_2 \\
 & | & \texttt{c()} \\
 & | & \texttt{c}(E_1, \ldots, E_n) \\
 & | & Var \leftarrow E \\
 & | & Var[E_{idx}] \\
 & | & Var[[E_{idx}]] \\
 & | & Var[E_{idx}] \leftarrow E \\
 & | & Var[[E_{idx}]] \leftarrow E \\
 & | & \texttt{setAttr}(Var, \{s_1, \ldots, s_n\}) \\
 & | & \texttt{getAttr}(Var)
\end{array}
$$

$\texttt{c(...)} \longrightarrow \oplus$ $\bowtie \longrightarrow$ **Recycle** $\texttt{v[i]} \longrightarrow$ **Select** $\texttt{v[i]} \leftarrow \texttt{e} \longrightarrow$ **Update**

Syntax

We omit usual `if-else` and `while` constructs for brevity.

$$
\begin{array}{rcl}
E_{idx} & ::= & \varepsilon \\
 & | & E
\end{array}
$$

$$
\begin{array}{rcl}
Atm & ::= & Var \mid Float \mid Int \mid Bool \mid \texttt{NULL} \\
Bool & ::= & \texttt{TRUE} \mid \texttt{FALSE} \mid \texttt{NA} \\
\bowtie_{arith} & ::= & + \mid - \mid * \mid / \\
\bowtie_{rel} & ::= & > \mid < \mid == \mid \neq \mid \leqslant \mid \geqslant \\
\bowtie_{log} & ::= & \& \mid | \\
\triangleright & ::= & ! \mid -
\end{array}
$$

$$
\begin{array}{rcl}
E & ::= & Atm \mid \triangleright E \\
 & | & e_1 \bowtie_{arith} e_2 \\
 & | & e_1 \bowtie_{rel} e_2 \\
 & | & e_1 \bowtie_{log} e_2 \\
 & | & \texttt{c()} \\
 & | & \texttt{c}(E_1, \ldots, E_n) \\
 & | & Var \leftarrow E \\
 & | & Var[E_{idx}] \\
 & | & Var[[E_{idx}]] \\
 & | & Var[E_{idx}] \leftarrow E \\
 & | & Var[[E_{idx}]] \leftarrow E \\
 & | & \texttt{setAttr}(Var, \{s_1, \ldots, s_n\}) \\
 & | & \texttt{getAttr}(Var)
\end{array}
$$

$\texttt{c}(\ldots) \longrightarrow \oplus \quad \bowtie \longrightarrow$ **Recycle** $\quad \texttt{v[i]} \longrightarrow$ **Select** $\quad \texttt{v[i]} \leftarrow \texttt{e} \longrightarrow$ **Update**

The concrete vector

$$Atm ::= Var \mid Float \mid Int \mid Bool \mid \texttt{NULL}$$

Definition Concrete Vector

A concrete vector $v \in \mathcal{V}$ is a pair $([e_1, ..., e_n], a) \in (\mathbb{N} \to \mathbb{V}_{Atm}) \times \mathcal{A}$: a total map from indices to atomic values, plus a set of attribute names.

$$([e_1, ..., e_n], \mathcal{A}) \equiv [e_1, ..., e_n]_{\mathcal{A}} \qquad ([e_1, ..., e_n], \emptyset) \equiv [e_1, ..., e_n]$$

The concrete vector

$$Atm ::= Var \mid Float \mid Int \mid Bool \mid \texttt{NULL}$$

Definition Concrete Vector

A concrete vector $v \in \mathcal{V}$ is a pair $([e_1, ..., e_n], a) \in (\mathbb{N} \to \mathbb{V}_{Atm}) \times \mathcal{A}$: a total map from indices to atomic values, plus a set of attribute names.

$$([e_1, ..., e_n], \mathcal{A}) \equiv [e_1, ..., e_n]_{\mathcal{A}} \qquad ([e_1, ..., e_n], \varnothing) \equiv [e_1, ..., e_n]$$

- Total means that every index has a value:
  - past the length it is `NULL` (the empty vector, not a missing element)
  - in-bounds, missingness is **NA**.

The concrete vector

$$Atm ::= Var \mid Float \mid Int \mid Bool \mid \texttt{NULL}$$

Definition — Concrete Vector

A concrete vector $v \in \mathcal{V}$ is a pair $([e_1, ..., e_n], a) \in (\mathbb{N} \to \mathbb{V}_{Atm}) \times \mathcal{A}$: a total map from indices to atomic values, plus a set of attribute names.

$$([e_1, ..., e_n], \mathcal{A}) \equiv [e_1, ..., e_n]_{\mathcal{A}} \qquad ([e_1, ..., e_n], \emptyset) \equiv [e_1, ..., e_n]$$

- Total means that every index has a value:
  - past the length it is `NULL` (the empty vector, not a missing element)
  - in-bounds, missingness is **NA**.
- We use the helper v^* for real readings:

$$v^*(i) \stackrel{\text{def}}{=} \begin{cases} v(i) \text{ if } v(i) \neq \texttt{NULL} \\ \textbf{NA} \text{ otherwise} \end{cases} \qquad \texttt{c}() \stackrel{\text{def}}{=} [\,] \qquad \texttt{c}(e_1, ..., e_n) \stackrel{\text{def}}{=} e_1 \oplus \cdots \oplus e_n$$

Recycling

UNIVERSITÀ DI PARMA

Definition Recycle

Recycle$(x, y) : \mathcal{V} \times \mathcal{V} \to \mathcal{V} \times \mathcal{V}$ is 3 steps:

- cast operands to their common type;

Recycling

UNIVERSITÀ DI PARMA

Definition Recycle

Recycle$(x, y) : \mathcal{V} \times \mathcal{V} \to \mathcal{V} \times \mathcal{V}$ is 3 steps:

- cast operands to their common type;
- length: $m = \max(|x|, |y|)$, decided by the operand lengths alone;

Definition Recycle

Recycle$(x, y) : \mathcal{V} \times \mathcal{V} \to \mathcal{V} \times \mathcal{V}$ is 3 steps:

- cast operands to their common type;
- length: $m = \max(|x|, |y|)$, decided by the operand lengths alone;
- content: ρ_c repeats the elements until they fill m:

$$\rho_c(v, l)(i) \stackrel{\text{def}}{=} v(((i - 1) \bmod |v|) + 1) \quad \text{for } i \in [1, l], \quad v \neq [\,]$$

Definition Recycle

Recycle$(x, y) : \mathcal{V} \times \mathcal{V} \to \mathcal{V} \times \mathcal{V}$ is 3 steps:

- cast operands to their common type;
- length: $m = \max(|x|, |y|)$, decided by the operand lengths alone;
- content: ρ_c repeats the elements until they fill m:

$$\rho_c(v, l)(i) \stackrel{\text{def}}{=} v(((i - 1) \bmod |v|) + 1) \quad \text{for } i \in [1, l], \quad v \neq [\,]$$

Example Unequal lengths

Recycle([0, 1], [3, 4, 5, 6]) = ([0, 1, 0, 1], [3, 4, 5, 6])

Selection

UNIVERSITÀ DI PARMA

The operator is **Select** : $\mathcal{V} \times \mathcal{V}_{Int} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: keep the positions listed, in the order given, duplicates included;

Selection

The operator is **Select** : $\mathcal{V} \times \mathcal{V}_{Int} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: keep the positions listed, in the order given, duplicates included;
- negative: drop the positions listed, keep the others in increasing order;

Selection

The operator is **Select** : $\mathcal{V} \times \mathcal{V}_{Int} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: keep the positions listed, in the order given, duplicates included;
- negative: drop the positions listed, keep the others in increasing order;
- logical: recycle the mask to **max(|v|, |c|)** and keep where it is **True**.

The operator is **Select** : $\mathcal{V} \times \mathcal{V}_{Int} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: keep the positions listed, in the order given, duplicates included;
- negative: drop the positions listed, keep the others in increasing order;
- logical: recycle the mask to **max(|v|, |c|)** and keep where it is **True**.

For example, the positive case:

Definition Positive Selection

$$\textbf{Select}(v, c) \stackrel{\text{def}}{=} \bigoplus_{i=1}^{|c|} \sigma_i \qquad \sigma_i \stackrel{\text{def}}{=} \begin{cases} [v^*(c(i))] & \text{if } c(i) > 0 \\ [\,] & \text{if } c(i) = 0 \\ [\textbf{NA}] & \text{if } c(i) = \textbf{NA} \end{cases}$$

Update

The operator is **Update** $(v, c, v_e) : \mathcal{V} \times \mathcal{V}_{Int} \times \mathcal{V} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: overwrite the positions c lists with the values v_e; past $|v|$ the vector grows;

Update

The operator is **Update** $(v, c, v_e) : \mathcal{V} \times \mathcal{V}_{Int} \times \mathcal{V} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: overwrite the positions c lists with the values v_e; past $|v|$ the vector grows;
- negative: overwrite all but the excluded positions; the length is unchanged;

Update

The operator is **Update** $(v, c, v_e) : \mathcal{V} \times \mathcal{V}_{Int} \times \mathcal{V} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: overwrite the positions c lists with the values v_e; past $|v|$ the vector grows;
- negative: overwrite all but the excluded positions; the length is unchanged;
- logical: recycle the mask to $\max(|v|, |c|)$ and overwrite where it is **True**.

Update

The operator is **Update** $(v, c, v_e) : \mathcal{V} \times \mathcal{V}_{Int} \times \mathcal{V} \rightharpoonup \mathcal{V}$, a partial function: the three modes are disjoint, and R rejects a selector that mixes them.

- positive: overwrite the positions c lists with the values v_e; past $|v|$ the vector grows;
- negative: overwrite all but the excluded positions; the length is unchanged;
- logical: recycle the mask to $\max(|v|, |c|)$ and overwrite where it is **True**.

For example, the positive case:

Definition Positive Update

$$\textbf{Update}\,(v, c, v_e) \stackrel{\text{def}}{=} r\big(u_1 \mapsto v'_e(1), \ldots, u_k \mapsto v'_e(k)\big) \qquad r = v \oplus \underbrace{[\textbf{NA}, \ldots, \textbf{NA}]}_{\max(C - |v|, 0)} \qquad v'_e = \rho_c(v_e, k)$$

with $C = \max\{c(i) \mid c(i) \neq \textbf{NA}\}$ and $u_1, \ldots, u_k$ the non-zero entries of c.

The running example

Four lines, five operators, one shape:

```
x <- c(TRUE, 2L, 3.0)         # Cast, then ⊕: ?
y <- x + c(10, 20)            # Recycle, then +: ?
z <- y[c(TRUE, FALSE, NA)]    # Select (logical): ?
z[c(2, 4)] <- c(50, 100)      # Update (positive): ?
```

Deterministic, and not evident. Part 5 has to predict exactly this, without running it.

The running example

Four lines, five operators, one shape:

```
x <- c(TRUE, 2L, 3.0)         # Cast, then ⊕: [1, 2, 3] double
y <- x + c(10, 20)            # Recycle, then +: ?
z <- y[c(TRUE, FALSE, NA)]    # Select (logical): ?
z[c(2, 4)] <- c(50, 100)      # Update (positive): ?
```

Deterministic, and not evident. Part 5 has to predict exactly this, without running it.

The running example

Four lines, five operators, one shape:

```
x <- c(TRUE, 2L, 3.0)         # Cast, then ⊕: [1, 2, 3] double
y <- x + c(10, 20)            # Recycle, then +: [11, 22, 13]
z <- y[c(TRUE, FALSE, NA)]    # Select (logical): ?
z[c(2, 4)] <- c(50, 100)      # Update (positive): ?
```

Deterministic, and not evident. Part 5 has to predict exactly this, without running it.

The running example

Four lines, five operators, one shape:

```
x <- c(TRUE, 2L, 3.0)         # Cast, then ⊕: [1, 2, 3] double
y <- x + c(10, 20)            # Recycle, then +: [11, 22, 13]
z <- y[c(TRUE, FALSE, NA)]    # Select (logical): [11, NA]
z[c(2, 4)] <- c(50, 100)      # Update (positive): ?
```

Deterministic, and not evident. Part 5 has to predict exactly this, without running it.

The running example

Four lines, five operators, one shape:

```
x <- c(TRUE, 2L, 3.0)         # Cast, then ⊕: [1, 2, 3] double
y <- x + c(10, 20)            # Recycle, then +: [11, 22, 13]
z <- y[c(TRUE, FALSE, NA)]    # Select (logical): [11, NA]
z[c(2, 4)] <- c(50, 100)      # Update (positive): [11, 50, NA, 100]
```

Deterministic, and not evident. Part 5 has to predict exactly this, without running it.

5. The Abstract Domain

A fixed prefix is not enough

Track two positions, summarise the rest.

```
x <- if (runif() >= 0.5) c(0, 0) else c(0, 0, 1, 100)   # length 2 or 4
```

A fixed prefix is not enough

Track two positions, summarise the rest.

```r
x <- if (runif() >= 0.5) c(0, 0) else c(0, 0, 1, 100)   # length 2 or 4
```

The naive element:

$$v = ([2, 4], \langle [0, 0][0, 0] \rangle, [1, 100], \textbf{AttrEmpty})$$

Positions 3 and 4 both read $[1, 100]$. We lose that `x[3]` is 1 whenever it exists, and that `x[4]` is 100 whenever it exists.

A fixed prefix is not enough

Track two positions, summarise the rest.

```r
x <- if (runif() >= 0.5) c(0, 0) else c(0, 0, 1, 100)   # length 2 or 4
```

The naive element:

$$v = ([2, 4], \langle [0, 0][0, 0] \rangle, [1, 100], \textbf{AttrEmpty})$$

Positions 3 and 4 both read **[1, 100]**. We lose that `x[3]` is 1 whenever it exists, and that `x[4]` is 100 whenever it exists.

A fixed prefix is not enough. When the length has a finite bound, every position is tracked; the summary is only for $u = +\infty$.

Raw abstract element

Definition

A *raw abstract vector* in $\tilde{\mathcal{V}}^{\sharp}_{\mathcal{B}}$ is a 4-tuple

$$v = \left([l, u], \langle p_1 \cdots p_k \rangle, s, a\right) \in \mathbf{Itv}_0 \times \mathcal{B}^{\sharp *} \times \mathcal{B}^{\sharp} \times \mathbf{Attr}^{\sharp}$$

with the invariant:

$$l \leq k \leq u \wedge \left(u \neq +\infty \Rightarrow \left(s = \bot_{\mathcal{B}^{\sharp}} \wedge u = k\right)\right)$$

Raw abstract element

Definition

A *raw abstract vector* in $\tilde{\mathcal{V}}^{\sharp}_{\mathcal{B}}$ is a 4-tuple

$$v = \left([l, u], \langle p_1 \cdots p_k \rangle, s, a\right) \in \mathbf{Itv}_0 \times \mathcal{B}^{\sharp *} \times \mathcal{B}^{\sharp} \times \mathbf{Attr}^{\sharp}$$

with the invariant:

$$l \leq k \leq u \wedge \left(u \neq +\infty \Rightarrow \left(s = \bot_{\mathcal{B}^{\sharp}} \wedge u = k\right)\right)$$

For example, setting $\mathcal{B}^{\sharp} = \mathbf{Itv}$:

- $([1, 2], \langle [1, 1][0, 0] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty}) \overset{\gamma_{\mathcal{V}^{\sharp}_{\mathbf{Itv}}}}{\longmapsto} \{[1], [1, 0]\}$
- $([1, +\infty], \langle [42, 42] \rangle, [42, 42], \mathbf{AttrEmpty}) \overset{\gamma_{\mathcal{V}^{\sharp}_{\mathbf{Itv}}}}{\longmapsto} \{[42], [42, 42], [42, 42, 42], \ldots\}$
- $([1, +\infty], \langle [42, 42][42, 42] \rangle, [42, 42], \mathbf{AttrEmpty}) \overset{\gamma_{\mathcal{V}^{\sharp}_{\mathbf{Itv}}}}{\longmapsto} \{[42], [42, 42], [42, 42, 42], \ldots\}$

UNIVERSITÀ DI PARMA

Definition

A *raw abstract vector* in $\tilde{\mathcal{V}}^{\sharp}_{\mathcal{B}}$ is a 4-tuple

$$v = \left([l, u], \langle p_1 \cdots p_k \rangle, s, a\right) \in \mathbf{Itv}_0 \times \mathcal{B}^{\sharp *} \times \mathcal{B}^{\sharp} \times \mathbf{Attr}^{\sharp}$$

with the invariant:

$$l \leq k \leq u \wedge \left(u \neq +\infty \Rightarrow \left(s = \bot_{\mathcal{B}^{\sharp}} \wedge u = k\right)\right)$$

For example, setting $\mathcal{B}^{\sharp} = \mathbf{Itv}$:

- $([1, 2], \langle [1, 1][0, 0] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty}) \overset{\gamma_{\mathcal{V}^{\sharp}_{\mathbf{Itv}}}}{\longmapsto} \{[1], [1, 0]\}$
- $([1, +\infty], \langle [42, 42] \rangle, [42, 42], \mathbf{AttrEmpty}) \overset{\gamma_{\mathcal{V}^{\sharp}_{\mathbf{Itv}}}}{\longmapsto} \{[42], [42, 42], [42, 42, 42], \ldots\}$
- $([1, +\infty], \langle [42, 42][42, 42] \rangle, [42, 42], \mathbf{AttrEmpty}) \overset{\gamma_{\mathcal{V}^{\sharp}_{\mathbf{Itv}}}}{\longmapsto} \{[42], [42, 42], [42, 42, 42], \ldots\}$

One set, many tuples

The last two tuples are the same element.

Definition Equivalence and minimum representative

- $v_1 \equiv v_2$ if they agree on L, s, a and on the known content up to the shorter sequence, with the longer one's extra positions equal to s.
- $v^{\downarrow}$ removes the trailing positions of the known content equal to s, keeping at least l.

One set, many tuples

The last two tuples are the same element.

Definition — Equivalence and minimum representative

- $v_1 \equiv v_2$ if they agree on L, s, a and on the known content up to the shorter sequence, with the longer one's extra positions equal to s.
- $v^{\downarrow}$ removes the trailing positions of the known content equal to s, keeping at least l.

$$([1, +\infty], \langle [42, 42] \rangle, [42, 42], \textbf{AttrEmpty})$$

$$([1, +\infty], \langle [42, 42][42, 42] \rangle, [42, 42], \textbf{AttrEmpty})$$

The second spells the same element: its extra $[42, 42]$ is s.

Every element is a class of raw tuples, plus two special elements.

Definition

The abstract domain is the lattice:

$$\mathcal{D}_{\mathcal{V}^\sharp_{\mathcal{B}}} = \left\langle \mathcal{V}^\sharp_{\mathcal{B}},\ \sqsubseteq,\ \sqcup,\ \sqcap,\ \bot_{\mathcal{V}^\sharp_{\mathcal{B}}},\ \top_{\mathcal{V}^\sharp_{\mathcal{B}}} \right\rangle$$

where $\mathcal{V}^\sharp_{\mathcal{B}} = \left\{ [v]_{\equiv} \;\middle|\; v \in \tilde{\mathcal{V}}^\sharp_{\mathcal{B}} \right\} \cup \left\{ \bot_{\mathcal{V}^\sharp_{\mathcal{B}}}, \top_{\mathcal{V}^\sharp_{\mathcal{B}}} \right\}$ and:

- $\bot_{\mathcal{V}^\sharp_{\mathcal{B}}} = \left(\bot_{\mathbf{Itv}_0}, \varepsilon, \bot_{\mathcal{B}^\sharp}, \bot_{\mathbf{Attr}^\sharp} \right)$
- $\top_{\mathcal{V}^\sharp_{\mathcal{B}}} = \left(\top_{\mathbf{Itv}_0}, \varepsilon, \top_{\mathcal{B}^\sharp}, \top_{\mathbf{Attr}^\sharp} \right)$

Order

$\sqsubseteq$ is containment: every vector the left element represents is represented by the right one.

Order

$\sqsubseteq$ is containment: every vector the left element represents is represented by the right one.

$$([2,2], \langle [1,1][2,2] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}) \sqsubseteq ([1,3], \langle [1,1][2,2][3,3] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

Order

$\sqsubseteq$ is containment: every vector the left element represents is represented by the right one.

$$([2,2], \langle [1,1][2,2] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}) \sqsubseteq ([1,3], \langle [1,1][2,2][3,3] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$([1,3], \langle [1,1][2,2][3,3] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}) \not\sqsubseteq ([2,2], \langle [1,1][2,2] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

Order

$\sqsubseteq$ is containment: every vector the left element represents is represented by the right one.

$$([2,2], \langle [1,1][2,2] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}) \sqsubseteq ([1,3], \langle [1,1][2,2][3,3] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$([1,3], \langle [1,1][2,2][3,3] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}) \not\sqsubseteq ([2,2], \langle [1,1][2,2] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

The left element of the second line also represents **[1]** and **[1, 2, 3]**; the right one does not.

Join

$\sqcup$ is the smallest element that contains both.

Join

$\sqcup$ is the smallest element that contains both.

$$([2,2], \langle [0,0][0,0] \rangle, \bot_{\textbf{ltv}}, \textbf{AttrEmpty}) \sqcup ([4,4], \langle [0,0][0,0][1,1][100,100] \rangle, \bot_{\textbf{ltv}}, \textbf{AttrEmpty})$$

Join

$\sqcup$ is the smallest element that contains both.

$$([2,2], \langle [0,0][0,0] \rangle, \bot_{\text{ltv}}, \textbf{AttrEmpty}) \sqcup ([4,4], \langle [0,0][0,0][1,1][100,100] \rangle, \bot_{\text{ltv}}, \textbf{AttrEmpty})$$
$$= ([2,4], \langle [0,0][0,0][1,1][100,100] \rangle, \bot_{\text{ltv}}, \textbf{AttrEmpty})$$

Join

$\sqcup$ is the smallest element that contains both.

$$([2,2], \langle [0,0][0,0] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty}) \sqcup ([4,4], \langle [0,0][0,0][1,1][100,100] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty})$$
$$= ([2,4], \langle [0,0][0,0][1,1][100,100] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty})$$

Both are below it, and positions 3 and 4 must stay apart: no smaller element contains both. This is the element a fixed prefix could not write.

Meet

$\sqcap$ is the largest element contained in both.

Meet

$\sqcap$ is the largest element contained in both.

$$([2,2], \langle [1,1][1,2] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty}) \sqcap ([1,+\infty], \langle [1,1][2,2][3,3] \rangle, [-1,-1], \mathbf{AttrEmpty})$$

Meet

$\sqcap$ is the largest element contained in both.

$$([2,2], \langle [1,1][1,2] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty}) \sqcap ([1,+\infty], \langle [1,1][2,2][3,3] \rangle, [-1,-1], \textbf{AttrEmpty})$$
$$= ([2,2], \langle [1,1][2,2] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty})$$

Meet

$\sqcap$ is the largest element contained in both.

$$([2,2], \langle [1,1][1,2] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty}) \sqcap ([1,+\infty], \langle [1,1][2,2][3,3] \rangle, [-1,-1], \textbf{AttrEmpty})$$
$$= ([2,2], \langle [1,1][2,2] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty})$$

Position 2 keeps only the values in both. Position 3 is impossible: the first element has length 2.

Meet

$\sqcap$ is the largest element contained in both.

$$([2,2], \langle [1,1][1,2] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty}) \sqcap ([1,+\infty], \langle [1,1][2,2][3,3] \rangle, [-1,-1], \textbf{AttrEmpty})$$
$$= ([2,2], \langle [1,1][2,2] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty})$$

Position 2 keeps only the values in both. Position 3 is impossible: the first element has length 2. When a position has no common value, the meet is $\bot_{\mathcal{V}_{\mathcal{B}}^{\sharp}}$:

$$([3,3], \langle [1,1][9,9][3,3] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty}) \sqcap ([3,3], \langle [1,1][2,2][3,3] \rangle, \bot_{\text{Itv}}, \textbf{AttrEmpty}) = \bot_{\mathcal{V}_{\mathcal{B}}^{\sharp}}$$

Widening

`x <- c(x, i)` in a loop appends one element per iteration: no finite bound.

Definition — Widening

- length: $[l', u'] = L_1 \nabla_{\mathbf{ltv}_0} L_2$
- content: $p_1(i) \nabla_{\mathcal{B}^\sharp} p_2(i)$ up to $\max(k_1, k_2)$; only the common prefix if $u' = +\infty \wedge (u_1 = +\infty \vee u_2 = +\infty)$.
- summary: $\bot_{\mathcal{B}^\sharp}$ if $u' \neq +\infty$; otherwise the value widening of s_1 with the join of s_2 and every position dropped from the known content.

Widening

`x <- c(x, i)` in a loop appends one element per iteration: no finite bound.

Definition Widening

- length: $[l', u'] = L_1 \nabla_{\mathbf{ltv}_0} L_2$
- content: $p_1(i) \nabla_{\mathcal{B}^\sharp} p_2(i)$ up to $\max(k_1, k_2)$; only the common prefix if $u' = +\infty \wedge (u_1 = +\infty \vee u_2 = +\infty)$.
- summary: $\bot_{\mathcal{B}^\sharp}$ if $u' \neq +\infty$; otherwise the value widening of s_1 with the join of s_2 and every position dropped from the known content.

$$x_0 = ([0, 0], \varepsilon, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$x_1 = x_0 \nabla_{\mathcal{V}^\sharp_{\mathbf{ltv}}} v_1 = ([0, +\infty], \langle [1, 1] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$x_2 = x_1 \nabla_{\mathcal{V}^\sharp_{\mathbf{ltv}}} v_2 = ([0, +\infty], \langle [1, 1] \rangle, [2, 2], \mathbf{AttrEmpty})$$

$$x_3 = x_2 \nabla_{\mathcal{V}^\sharp_{\mathbf{ltv}}} v_3 = ([0, +\infty], \langle [1, 1] \rangle, [2, +\infty], \mathbf{AttrEmpty}) = x_4$$

Definition Concretization

The concretization function $\gamma_{\mathcal{V}^\sharp_\mathcal{B}} : \mathcal{V}^\sharp_\mathcal{B} \to \mathcal{P}(\mathcal{V})$ is defined as:

$$\gamma_{\mathcal{V}^\sharp_\mathcal{B}}(([l,u],\boldsymbol{p},s,a)) \stackrel{\text{def}}{=} \left\{ [c_1,\ldots,c_n]_\mathcal{A} \in \mathcal{V} \;\middle|\; \begin{array}{l} n \in \gamma_{\mathbf{Itv}_0}([l,u]),\ \mathcal{A} \in \gamma_{\mathbf{Attr}^\sharp}(a), \\ \forall i \in [1,\min(n,|\boldsymbol{p}|)] : c_i \in \gamma_{\mathcal{B}^\sharp}(p_i), \\ \forall j \in [|\boldsymbol{p}|+1,n] : c_j \in \gamma_{\mathcal{B}^\sharp}(s) \end{array} \right\}$$

The analysis is concretization-based: it relies solely on γ, since no vector-level best abstraction exists.

Why There Is No Galois Connection

A Galois connection would give every concrete element a best abstraction, the least d with $c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d)$:

$$\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c) = \sqcap \left\{ d \;\middle|\; c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d) \right\}$$

Why There Is No Galois Connection

A Galois connection would give every concrete element a best abstraction, the least d with $c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d)$:

$$\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c) = \sqcap \left\{ d \;\middle|\; c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d) \right\}$$

The two sides of

$$\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c) \sqsubseteq d \iff c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d)$$

are:

- Soundness: $\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c) \sqsubseteq d \implies c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d)$. At $d = \alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c)$ this gives $c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}\left(\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c)\right)$, so $\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c)$ is an upper bound of c.
- Best abstraction: $c \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(d) \implies \alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c) \sqsubseteq d$. Every upper bound is above $\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c)$: the least one exists and is $\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(c)$.

Why There Is No Galois Connection

One set breaks it. Take $\mathcal{C} = \{[1], [1,2], [1,2,3], \ldots\}$ and, for every $i \geq 1$,

$$v_i = ([1, +\infty], \langle [1,1] \cdots [i,i] \rangle, [i+1, +\infty], \textbf{AttrEmpty})$$

Why There Is No Galois Connection

One set breaks it. Take $\mathcal{C} = \{[1], [1,2], [1,2,3], \ldots\}$ and, for every $i \geq 1$,

$$v_i = ([1, +\infty], \langle [1,1] \cdots [i,i] \rangle, [i+1, +\infty], \textbf{AttrEmpty})$$

- $\mathcal{C} \subseteq \gamma_{\mathcal{V}^\sharp_{\mathcal{B}}}(v_i)$ for all i: the first i positions are exact, the tail in $[i+1, +\infty]$.

Why There Is No Galois Connection

One set breaks it. Take $\mathcal{C} = \{[1], [1,2], [1,2,3], \ldots\}$ and, for every $i \geq 1$,

$$v_i = ([1, +\infty], \langle [1,1] \cdots [i,i] \rangle, [i+1, +\infty], \textbf{AttrEmpty})$$

- $\mathcal{C} \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(v_i)$ for all i: the first i positions are exact, the tail in $[i+1, +\infty]$.
- $v_{i+1} \sqsubseteq v_i$: one more exact position and a tighter summary.

Why There Is No Galois Connection

One set breaks it. Take $\mathcal{C} = \{[1], [1,2], [1,2,3], \ldots\}$ and, for every $i \geq 1$,

$$v_i = ([1, +\infty], \langle [1,1] \cdots [i,i] \rangle, [i+1, +\infty], \textbf{AttrEmpty})$$

- $\mathcal{C} \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(v_i)$ for all i: the first i positions are exact, the tail in $[i+1, +\infty]$.
- $v_{i+1} \sqsubseteq v_i$: one more exact position and a tighter summary.
- Any upper bound with k exact positions is refined by v_{k+1}: hence no minimum.

Why There Is No Galois Connection

One set breaks it. Take $\mathcal{C} = \{[1], [1,2], [1,2,3], \ldots\}$ and, for every $i \geq 1$,

$$v_i = ([1, +\infty], \langle [1,1] \cdots [i,i] \rangle, [i+1, +\infty], \textbf{AttrEmpty})$$

- $\mathcal{C} \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(v_i)$ for all i: the first i positions are exact, the tail in $[i+1, +\infty]$.
- $v_{i+1} \sqsubseteq v_i$: one more exact position and a tighter summary.
- Any upper bound with k exact positions is refined by v_{k+1}: hence no minimum.
- The meet would need $\langle [1,1][2,2] \ldots \rangle$, which no finite tuple holds: no Galois connection.

Why There Is No Galois Connection

One set breaks it. Take $\mathcal{C} = \{[1], [1,2], [1,2,3], \ldots\}$ and, for every $i \geq 1$,

$$v_i = ([1, +\infty], \langle [1,1] \cdots [i,i] \rangle, [i+1, +\infty], \textbf{AttrEmpty})$$

- $\mathcal{C} \subseteq \gamma_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(v_i)$ for all i: the first i positions are exact, the tail in $[i+1, +\infty]$.
- $v_{i+1} \sqsubseteq v_i$: one more exact position and a tighter summary.
- Any upper bound with k exact positions is refined by v_{k+1}: hence no minimum.
- The meet would need $\langle [1,1][2,2] \ldots \rangle$, which no finite tuple holds: no Galois connection.

A lattice, not complete: the chain has no infimum, so $\alpha_{\mathcal{V}_{\mathcal{B}}^{\sharp}}(\mathcal{C})$ does not exist. A lower bound fixes only finitely many positions, so the infimum would be $\mathcal{C}$ itself.

6. Abstract Operators

The operator map

Every abstract operator belongs to one of three families:

- Data movement: concatenation and recycling put vectors together or repeat them.
- Access: selection reads positions, update writes them.
- Metadata: coercion changes the type, attributes annotate the vector.

The operator map

Every abstract operator belongs to one of three families:

- Data movement: concatenation and recycling put vectors together or repeat them.
- Access: selection reads positions, update writes them.
- Metadata: coercion changes the type, attributes annotate the vector.

Three problems generate every difficulty:

- The zero index is silently dropped.
- Index values are known only at run time.
- A negative index says what is not there.

Coercion and Attributes

The type is a finite component: the chain **Bool** < **Int** < **Float**, and the typed domain is $\mathcal{T} \times \mathcal{V}_{\mathcal{B}}^{\sharp}$.

Coercion and Attributes

The type is a finite component: the chain **Bool** < **Int** < **Float**, and the typed domain is $\mathcal{T} \times \mathcal{V}_{\mathcal{B}}^{\sharp}$.

```
τ(c(TRUE, 3L, 2.5))
```

Coercion and Attributes

The type is a finite component: the chain **Bool** < **Int** < **Float**, and the typed domain is $\mathcal{T} \times \mathcal{V}_{\mathcal{B}}^{\sharp}$.

$$\tau(\texttt{c(TRUE, 3L, 2.5)}) = (\{\textbf{Bool}\} \sqcup \{\textbf{Int}\}) \sqcup \{\textbf{Float}\}$$
$$= \{\textbf{Int}\} \sqcup \{\textbf{Float}\} = \{\textbf{Float}\}$$

Three arguments, two joins: the abstract type is **Float**.

Coercion and Attributes

The type is a finite component: the chain **Bool** < **Int** < **Float**, and the typed domain is $\mathcal{T} \times \mathcal{V}_{\mathcal{B}}^{\sharp}$.

$$\tau(\texttt{c(TRUE, 3L, 2.5)}) = (\{\textbf{Bool}\} \sqcup \{\textbf{Int}\}) \sqcup \{\textbf{Float}\}$$
$$= \{\textbf{Int}\} \sqcup \{\textbf{Float}\} = \{\textbf{Float}\}$$

Three arguments, two joins: the abstract type is **Float**.

Attributes are a (**MustHave**, **MayHave**) pair: **setAttr**$^{\sharp}$ pins it to (A, A), **getAttr**$^{\sharp}$ projects.

Example

`m <- matrix(1:2, nrow = 1)` gives `dim` and `class`:

$$v = \left([2,2], \langle [1,1][2,2] \rangle, \bot_{\textbf{Itv}}, (\{\textbf{dim}, \textbf{class}\}, \{\textbf{dim}, \textbf{class}\})\right)$$

Concatenation

Concatenation joins the lengths and concatenates the known contents.

Example

$$v_1 = ([1,2], \langle [0,0][1,1] \rangle, \bot_{\mathsf{Itv}}, \mathbf{AttrEmpty})$$
$$v_2 = ([1,1], \langle [42,42] \rangle, \bot_{\mathsf{Itv}}, \mathbf{AttrEmpty})$$

Concatenation

Concatenation joins the lengths and concatenates the known contents.

Example

$$v_1 = ([1,2], \langle [0,0][1,1] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$
$$v_2 = ([1,1], \langle [42,42] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

The concretizations are $\gamma(v_1) = \{[0], [0,1]\}$ and $\gamma(v_2) = \{[42]\}$; the possible concatenations are $\{[0,42], [0,1,42]\}$.

Concatenation

Concatenation joins the lengths and concatenates the known contents.

Example

$$v_1 = ([1,2], \langle [0,0][1,1] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

$$v_2 = ([1,1], \langle [42,42] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

The concretizations are $\gamma(v_1) = \{[0], [0,1]\}$ and $\gamma(v_2) = \{[42]\}$; the possible concatenations are $\{[0,42], [0,1,42]\}$.

$$v_1 \oplus^\sharp v_2 = ([2,3], \langle [0,0][1,42][42,42] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

Recycling: two uncertainties

$$v_i = \left([l_i, u_i], \left\langle p_{i1} \cdots p_{i\,|\boldsymbol{p}_i|} \right\rangle, s_i, a_i \right)$$

$$\textbf{Recycle}^\sharp : \mathcal{V}_{\mathcal{B}}^\sharp \times \mathcal{V}_{\mathcal{B}}^\sharp \to \mathcal{V}_{\mathcal{B}}^\sharp \times \mathcal{V}_{\mathcal{B}}^\sharp$$

The resulting recycled length is $[l', u'] = \left[\max(l_1, l_2), \max(u_1, u_2)\right]$.

$$v_i' = \left([l', u'], \bigsqcup_{d=l'}^{\max(|\boldsymbol{p}_1|, |\boldsymbol{p}_2|)}{}_{\mathcal{B}^{\sharp *}} \rho_f^\sharp(\boldsymbol{p}_i, s_i, l_i, d),\ s_i',\ a_i \right)$$

Recycling: two uncertainties

$$v_i = \left([l_i, u_i], \left\langle p_{i1} \cdots p_{i\,|\boldsymbol{p}_i|} \right\rangle, s_i, a_i \right)$$

$$\textbf{Recycle}^\sharp : \mathcal{V}_{\mathcal{B}}^\sharp \times \mathcal{V}_{\mathcal{B}}^\sharp \to \mathcal{V}_{\mathcal{B}}^\sharp \times \mathcal{V}_{\mathcal{B}}^\sharp$$

The resulting recycled length is $[l', u'] = \left[\max(l_1, l_2), \max(u_1, u_2)\right]$.

$$v_i' = \left([l', u'], \bigsqcup_{d=l'}^{\max(|\boldsymbol{p}_1|, |\boldsymbol{p}_2|)}{}_{\mathcal{B}^{\sharp *}} \rho_f^\sharp(\boldsymbol{p}_i, s_i, l_i, d),\ s_i',\ a_i \right)$$

- The target length d lives in $\gamma_{\textbf{Itv}}([l', \max(|\boldsymbol{p}_1|, |\boldsymbol{p}_2|)])$ → join over $\rho_f^\sharp$.

Recycling: two uncertainties

$$v_i = \left([l_i, u_i], \left\langle p_{i1} \cdots p_{i\,|\boldsymbol{p}_i|} \right\rangle, s_i, a_i \right)$$

$$\textbf{Recycle}^\sharp : \mathcal{V}_{\mathcal{B}}^\sharp \times \mathcal{V}_{\mathcal{B}}^\sharp \to \mathcal{V}_{\mathcal{B}}^\sharp \times \mathcal{V}_{\mathcal{B}}^\sharp$$

The resulting recycled length is $[l', u'] = \left[\max(l_1, l_2), \max(u_1, u_2)\right]$.

$$v_i' = \left([l', u'], \bigsqcup_{d=l'}^{\max(|\boldsymbol{p}_1|, |\boldsymbol{p}_2|)}{}_{\mathcal{B}^{\sharp *}} \rho_f^\sharp(\boldsymbol{p}_i, s_i, l_i, d),\ s_i',\ a_i \right)$$

- The target length d lives in $\gamma_{\textbf{Itv}}([l', \max(|\boldsymbol{p}_1|, |\boldsymbol{p}_2|)]) \to$ join over $\rho_f^\sharp$.
- At a fixed d, the original length ranges over $[l_i, u_i]$, so $i \bmod k$ changes $\to$ join over $\rho_c^\sharp$.

Recycling: an example

UNIVERSITÀ DI PARMA

Example

Recycle $\boldsymbol{p} = \langle [1,1][2,2] \rangle$ to length 3, with original length 1 or 2:

$$\rho^{\sharp}_{f}(\boldsymbol{p}, \bot_{\mathbf{ltv}}, 1, 3) = \rho^{\sharp}_{c}(\boldsymbol{p}, 1, 3) \sqcup_{\mathbf{ltv}^*} \rho^{\sharp}_{c}(\boldsymbol{p}, 2, 3)$$

Recycling: an example

UNIVERSITÀ DI PARMA

Example

Recycle $\boldsymbol{p} = \langle [1,1][2,2] \rangle$ to length 3, with original length 1 or 2:

$$\begin{aligned} \rho_f^\sharp(\boldsymbol{p}, \bot_{\mathbf{ltv}}, 1, 3) &= \rho_c^\sharp(\boldsymbol{p}, 1, 3) \sqcup_{\mathbf{ltv}^*} \rho_c^\sharp(\boldsymbol{p}, 2, 3) \\ &= \langle [1,1][1,1][1,1] \rangle \sqcup_{\mathbf{ltv}^*} \langle [1,1][2,2][1,1] \rangle \end{aligned}$$

Recycling: an example

UNIVERSITÀ DI PARMA

Example

Recycle $\boldsymbol{p} = \langle [1,1][2,2] \rangle$ to length 3, with original length 1 or 2:

$$\rho^{\sharp}_{f}(\boldsymbol{p}, \bot_{\mathbf{ltv}}, 1, 3) = \rho^{\sharp}_{c}(\boldsymbol{p}, 1, 3) \sqcup_{\mathbf{ltv}^*} \rho^{\sharp}_{c}(\boldsymbol{p}, 2, 3)$$

$$= \langle [1,1][1,1][1,1] \rangle \sqcup_{\mathbf{ltv}^*} \langle [1,1][2,2][1,1] \rangle = \langle [1,1][1,2][1,1] \rangle$$

Selection

$\mathbf{Select}^\sharp : \mathcal{V}^\sharp_{\mathcal{B}} \times \mathcal{V}^\sharp_{\mathbf{Itv}} \to \mathcal{V}^\sharp_{\mathcal{B}}$

R forbids mixed indices: the sign of the selector chooses the branch.

Positive $\mathbf{Select}^\sharp_{\mathbf{pos}}$
Indices ≥ 0 or NA. Extracts by position.

Negative $\mathbf{Select}^\sharp_{\mathbf{neg}}$
Indices < 0. Excludes by position.

Logical $\mathbf{Select}^\sharp_{\mathbf{log}}$
Mask in $\{\mathbf{NA}, \mathbf{True}, \mathbf{False}\}^+$. Selects where true.

The selector is filtered by the sign of its concretization: abstract filtering, the same move as filtering a guard by the branch condition.

$$\boldsymbol{p}_2^+ = \langle p \in \boldsymbol{p}_2 \mid \gamma_{\mathbf{Itv}}(p) \subseteq \mathbb{Z}_{\geq 0} \cup \{\mathbf{NA}\} \rangle$$

$$\boldsymbol{p}_2^- = \langle p \in \boldsymbol{p}_2 \mid \gamma_{\mathbf{Itv}}(p) \subseteq \mathbb{Z}_{< 0} \rangle$$

$$\mathbf{Select}^\sharp(v_1, v_2) = \mathbf{Select}^\sharp_{\mathbf{pos}}(v_1, v_2^+) \sqcup \mathbf{Select}^\sharp_{\mathbf{neg}}(v_1, v_2^-)$$

Selection

$\mathbf{Select}^\sharp : \mathcal{V}^\sharp_{\mathcal{B}} \times \mathcal{V}^\sharp_{\mathbf{Itv}} \to \mathcal{V}^\sharp_{\mathcal{B}}$

R forbids mixed indices: the sign of the selector chooses the branch.

Positive $\mathbf{Select}^\sharp_{\mathbf{pos}}$
Indices ≥ 0 or NA. Extracts by position.

Negative $\mathbf{Select}^\sharp_{\mathbf{neg}}$
Indices < 0. Excludes by position.

Logical $\mathbf{Select}^\sharp_{\mathbf{log}}$
Mask in $\{\mathbf{NA}, \mathbf{True}, \mathbf{False}\}^+$. Selects where true.

The selector is filtered by the sign of its concretization: abstract filtering, the same move as filtering a guard by the branch condition.

$$\boldsymbol{p}_2^+ = \left\langle p \in \boldsymbol{p}_2 \;\middle|\; \gamma_{\mathbf{Itv}}(p) \subseteq \mathbb{Z}_{\geq 0} \cup \{\mathbf{NA}\} \right\rangle$$

$$\boldsymbol{p}_2^- = \left\langle p \in \boldsymbol{p}_2 \;\middle|\; \gamma_{\mathbf{Itv}}(p) \subseteq \mathbb{Z}_{<0} \right\rangle$$

$$\mathbf{Select}^\sharp(v_1, v_2) = \mathbf{Select}^\sharp_{\mathbf{pos}}(v_1, v_2^+) \sqcup \mathbf{Select}^\sharp_{\mathbf{neg}}(v_1, v_2^-)$$

Selection: the positive case

AdjustForZeros handles the zero index: it drops definite zeros and joins the indices that may land on the same position. A position too wide to enumerate collapses the source to **Squash**(*v*).

Selection: the positive case

AdjustForZeros handles the zero index: it drops definite zeros and joins the indices that may land on the same position. A position too wide to enumerate collapses the source to **Squash**(v).

Example

$$v = ([2,3], \langle [10,10][20,20][30,30] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$s = ([2,3], \langle [1,2][0,1][1,100] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$\mathbf{AdjustForZeros}(s) = ([1,3], \langle [1,2][0,100][1,100] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$\mathbf{Select}^{\sharp}_{\mathbf{pos}}(v,s) = \Big([1,3], \Big\langle [10,20], [10,30] \sqcup \alpha_{\mathbf{ltv}}(\mathbf{NA}), [10,30] \sqcup \alpha_{\mathbf{ltv}}(\mathbf{NA}) \Big\rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}\Big)$$

Position 1 joins 1 and 2: **[10, 20]**; positions 2 and 3 are too wide to enumerate, so each is **Squash**(v) $\sqcup$ $\alpha_{\mathbf{ltv}}$(**NA**).

Selection: exclusion and masks

Negative selection excludes: when a possibly-deleted set covers a range, the survivors cannot be listed one by one, so they collapse to **SquashExcept**(v, **MustDeleted**).

Example

$$v = ([4,4], \langle [10,10][20,20][30,30][40,40] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

$$s = ([2,2], \langle [-1,-1][-3,-3] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

$$\mathbf{MustDeleted} = \{1,3\}, \quad \mathbf{MayDeleted} = \{1,3\}, \quad \mathbf{MustNotDeleted} = \{2,4\}$$

$$\mathbf{Select}^{\sharp}_{\mathbf{neg}}(v,s) = ([2,2], \langle [20,20][40,40] \rangle, \bot_{\mathbf{Itv}}, \mathbf{AttrEmpty})$$

Selection: exclusion and masks

Logical selection reads a boolean mask.

Example

$$v = ([4,4], \langle [10,10][20,20][30,30][40,40] \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$s = ([4,4], \langle \mathbf{True}, \mathbf{False}, \mathbf{NA}, \mathbf{True} \rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty})$$

$$\mathbf{Select}^{\sharp}_{\mathbf{log}}(v,s) = \left([3,3], \left\langle [10,10], \alpha_{\mathbf{ltv}}(\mathbf{NA}), [40,40] \right\rangle, \bot_{\mathbf{ltv}}, \mathbf{AttrEmpty}\right)$$

The mask is recycled with $\rho^{\sharp}_{f}$: **True** selects, **False** skips, NA gives $\alpha_{\mathbf{ltv}}(\mathbf{NA})$; the length subtracts the definite **False**.

Both are the positive operator plus a set of positions: the complement of the deleted set, or the positions where the mask is true.

Update: the positive case and its cost

Update is selection plus a write: same filtering, same AdjustForZeros.

- Non-enumerable selector: collapse to $\mathbf{Squash}(v) \sqcup \mathbf{Squash}(v_e)$; length unbounded.
- Infinite, enumerable: weak update; a position may be written twice.
- Finite: extend to the largest index with NA, then write.

Example

$$v = ([5,5], \langle [1,1] \cdots [5,5] \rangle)$$

$$s = ([1,1], \langle [10,10] \rangle), \quad v_e = ([1,1], \langle [42,42] \rangle)$$

$$\mathbf{Update}^{\sharp}_{\mathbf{pos}}(v, s, v_e) = \left([10,10], \left\langle [1,1] \cdots [5,5]\ \alpha_{\mathsf{Itv}}(\mathbf{NA}) \cdots \alpha_{\mathsf{Itv}}(\mathbf{NA})\ [42,42] \right\rangle\right)$$

Ten positions, four NA. An update at index i materialises i positions, and widening closes the tail.

Update: exclusion and masks

Negative update has three sets: not updated for sure, may not be updated, updated for sure. The third writes strongly; the length does not change (`v[-2] ← 5` writes, it does not delete).

Example

$$v = ([4,4], \langle [10,10][20,20][30,30][40,40] \rangle)$$

$$s = ([1,1], \langle [-2,-2] \rangle), \quad v_e = ([1,1], \langle [99,99] \rangle)$$

$$\mathbf{Update}^{\sharp}_{\mathbf{neg}}(v, s, v_e) = ([4,4], \langle [10,10][99,99][30,30][40,40] \rangle)$$

Update: exclusion and masks

Logical update recycles mask and values: true writes strongly, maybe-true weakly, false keeps.

Example

$$v = ([4,4], \langle [10,10][20,20][30,30][40,40] \rangle)$$

$$s = \left([4,4], \left\langle [1,1]^{1}, [0,0]^{2}, [1,1], [0,0] \right\rangle\right)$$

$$\textbf{Update}^{\sharp}_{\textbf{log}}(v, s, \langle [99,99] \rangle) = ([4,4], \langle [99,99][20,20][99,99][40,40] \rangle)$$

The one refusal: if the mask may contain NA and the replacement may be longer than 1 ($u_3 \neq 1$), the concrete program is an error and the abstract returns $\bot_{\mathcal{V}^{\sharp}_{\mathcal{B}}}$.

Update is selection plus a write; the difficulty is knowing where.

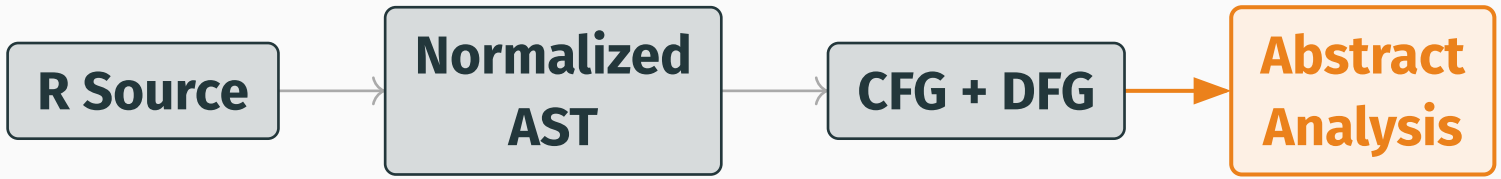
[1] $\alpha_{\text{ltv}}(\textbf{True})$
[2] $\alpha_{\text{ltv}}(\textbf{False})$

7. Implementation and Validation

Pipeline and visitor

The domain is integrated in flowR. The orange stage is the contribution of this work:

- Worklist fixpoint: join at merge points, widening $\nabla_{\mathcal{V}_{\mathcal{B}}^{\sharp}}$ at loops.
- The visitor decomposes the state into length, known content, summary, attributes and type, then dispatches every CFG vertex to the corresponding abstract operator.
- Outside µR it falls back to $\top_{\mathcal{V}_{\mathcal{B}}^{\sharp}}$: sound on µR, conservative elsewhere.



Three tests

Recycling

```
if (*) v <- c(1, 2, 3) else v <- c(0, 1)
r <- v + c(9, 10, 11)
```

↳ $r = ([3,3], \langle [9,10], [11,12], [11,14] \rangle, \bot_{\text{ltv}})$

Uncertainty

```
v <- *; s <- c(1, 2, 3)
x <- s[v]; y <- x[c(1, 2)]
```

↳ $* = \top_{\mathcal{V}^\sharp_{\text{ltv}}}, y = \left([2,2], \left\langle [1,3] \sqcup_{\text{ltv}} \alpha_{\text{ltv}}(\textbf{NA}), [1,3] \sqcup_{\text{ltv}} \alpha_{\text{ltv}}(\textbf{NA}) \right\rangle, \bot_{\text{ltv}}\right)$

Loop

```
x <- c(1, 2, 3); i <- 1
while (*) { x <- c(x, i); i <- i + 1 }; r <- x
```

↳ $r = \left([3, +\infty], \langle [1,1], [2,2], [3,3] \rangle, [1, +\infty] \sqcup_{\text{ltv}} \alpha_{\text{ltv}}(\textbf{NA})\right)$

What the numbers say

- 61 programs, 6 categories: ^{14}Creation, ^{9}Recycling, ^{12}Select, ^{12}Update, ^{8}Uncertainty, ^{6}Loops.

What the numbers say

- 61 programs, 6 categories: ^{14}Creation, ^{9}Recycling, ^{12}Select, ^{12}Update, ^{8}Uncertainty, ^{6}Loops.
- 2 checks:
  - the computed abstract value equals the one derived by hand from the definitions;
  - where a concrete run exists, the concrete R run lies in its concretization.

What the numbers say

- 61 programs, 6 categories: [14]Creation, [9]Recycling, [12]Select, [12]Update, [8]Uncertainty, [6]Loops.
- 2 checks:
  - the computed abstract value equals the one derived by hand from the definitions;
  - where a concrete run exists, the concrete R run lies in its concretization.
- Cost: an update at index *i* materialises *i* positions; widening closes the tail.

What the numbers say

- 61 programs, 6 categories: [14]Creation, [9]Recycling, [12]Select, [12]Update, [8]Uncertainty, [6]Loops.
- 2 checks:
  - the computed abstract value equals the one derived by hand from the definitions;
  - where a concrete run exists, the concrete R run lies in its concretization.
- Cost: an update at index *i* materialises *i* positions; widening closes the tail.
- Limits: non-relational; no length relations between variables; no interprocedural analysis.

8. Conclusions and Open Problems

Contributions and proof status

Built

- µR: the vector features we analyze, made explicit.
- A parametric domain: length, known prefix, summary, attributes.
- Abstract operators: recycling, selection, update.
- A prototype in flowR: 61 programs, 61/61.

Contributions and proof status

UNIVERSITÀ DI PARMA

Built

- μR: the vector features we analyze, made explicit.
- A parametric domain: length, known prefix, summary, attributes.
- Abstract operators: recycling, selection, update.
- A prototype in flowR: 61 programs, 61/61.

Proved

- Partial order and meet.
- Widening termination.
- Recycling and positive selection.

Empirical only

- Negative and logical selection.
- All updates.
- Attributes.

Open problems

Where is each gap: domain, semantics, or engineering?

Length relations between variables: reduced product	domain
Branch sensitivity: disjunctive states at merge points	domain

Open problems

Where is each gap: domain, semantics, or engineering?

Length relations between variables: reduced product	domain
Branch sensitivity: disjunctive states at merge points	domain
Interprocedural analysis under lazy evaluation	semantics
S3/S4 dispatch modelled, not over-approximated	semantics

Open problems

Where is each gap: domain, semantics, or engineering?

Length relations between variables: reduced product	domain
Branch sensitivity: disjunctive states at merge points	domain
Interprocedural analysis under lazy evaluation	semantics
S3/S4 dispatch modelled, not over-approximated	semantics
The remaining soundness proofs	engineering
Precision and cost on real CRAN packages	engineering

Open problems

Where is each gap: domain, semantics, or engineering?

Length relations between variables: reduced product	domain
Branch sensitivity: disjunctive states at merge points	domain
Interprocedural analysis under lazy evaluation	semantics
S3/S4 dispatch modelled, not over-approximated	semantics
The remaining soundness proofs	engineering
Precision and cost on real CRAN packages	engineering

M. Di Agostino, F. Sihler, V. Arceri, O. Gerstl, M. Tichy. *Towards Statically Reasoning about R Vectors*. NSAD '26, pp. 18–25. doi:10.1145/3840563.3843708

Slides available